Praktikum Web-Applikationen · WS 2026/27

Prof. Dr. Pascal Laube · HTWG Konstanz

Überblick

Im Praktikum entwickelt ihr **in Einzelarbeit** über das ganze Semester eine eigene Full-Stack-Web-Applikation. Das **Thema wählt ihr selbst**: Rezeptbuch, Vereinsplaner, Lernkarten, Ticketing, Tauschbörse oder etwas ganz anderes. Wichtig ist, dass die App echte Daten verwaltet und mehrere Nutzerinnen und Nutzer mit unterschiedlichen Rechten hat.

Eure App wächst mit der Vorlesung. Zu jedem Meilenstein müssen die Konzepte der zugehörigen Vorlesungen **im Code sichtbar** sein, nicht nur installiert, sondern eingesetzt.

Themenidee: In der Übung am **08.10.** bespreche ich mit jeder und jedem von euch kurz das Thema. Bringt dafür eine Idee mit (zwei, drei Sätze: Was kann die App? Für wen ist sie? Welche Rollen gibt es?).

Meilensteine

Alle vier Meilensteine sind **Pflichtabgaben** und Voraussetzung für die Projektnote.

Meilenstein	Inhalt	Abgabe
M1	Projektstart und Fundament: HTML/CSS-Prototyp	**Mi 21.10.2026, 23:59**
M2	React-App: Tooling, Komponenten, Hooks, Routing	**Mi 18.11.2026, 23:59**
M3	Backend, API, Datenbank, Authentifizierung mit Rollen, Security-Check	**Mi 13.01.2027, 23:59**
M4	Abschluss: Tests, Betrieb, Demo-Video, Ausarbeitung	nach Vorlesungsende, Termin wird angekündigt

Abgabe pro Meilenstein: Das **README.md** eures Repositorys als Textabgabe in Moodle. Es enthält den Repository-Link und eine **Kriterien-Zuordnung**: Welches Kriterium ist in welcher Datei und Zeile erfüllt? Der abgegebene Stand ist im Repository mit einem **Git-Tag** markiert (`Meilenstein 1` bis `Meilenstein 4`). Mein GitHub-Account `PascalLaube` braucht als Collaborator Lesezugriff.

M1 · Projektstart und Fundament

Abgabe: Mi 21.10.2026, 23:59 · **VL-Bezug:** VL 2 und 3 (Recap HTML und CSS, Das Web und seine Protokolle)

Kriterium	Beschreibung
Repository	Eigenes Git-Repo, regelmäßige Commits. README mit Projektidee (2–3 Sätze), Name, Matrikelnummer, Repo-Link und Kriterien-Zuordnung. Name und Matrikelnummer nur bei privatem Repository, sonst nur in der Textabgabe in Moodle.
HTML	Semantische Struktur: `header`, `main`, `nav`, `article`, `section`, `footer`. Mindestens ein sinnvolles Formular mit `label for` und `name`.
CSS	Responsives Layout mit Flexbox oder Grid, konsistente Typografie und Farbgebung, mindestens eine Media Query.
HTTP-Bewusstsein	Klare URL-Struktur für Seiten und Ressourcen.

Ergebnis: Ein HTML/CSS-Prototyp eurer App-Idee, statisch, aber mit durchdachter Struktur und ansprechendem Layout. Kein JavaScript nötig. Details in der Moodle-Aufgabe M1.

M2 · React-App

Abgabe: Mi 18.11.2026, 23:59 · **VL-Bezug:** VL 4 und 5 (JavaScript kompakt, React kompakt)

Kriterium	Beschreibung
Tooling	Projekt mit **npm** und **Vite**, **TypeScript** aktiv genutzt, nicht nur konfiguriert. `npm install && npm run dev` ist dokumentiert und funktioniert.
Komponenten	Sinnvolle Zerlegung in Komponenten, Daten über Props.
State und Hooks	`useState` für lokalen Zustand, `useEffect` wo sinnvoll. Mindestens eine durchgängige Nutzeraktion (z. B. Formular → Liste, Filter).
Routing	**React Router** mit mindestens drei Routen (z. B. Übersicht, Detail, Formular).

Ergebnis: Euer Prototyp aus M1 lebt jetzt als React/TypeScript-App mit echter Interaktion. Daten dürfen noch aus einer lokalen Datei oder dem State kommen.

M3 · Backend, Daten, Authentifizierung und Sicherheit

Abgabe: Mi 13.01.2027, 23:59 · **VL-Bezug:** VL 6 bis 12 (Browser, API-Design, Backend-Architektur, Persistenz, Zustand, Identität, Web Security)

Kriterium	Beschreibung
Backend und API	Eigener Server (z. B. Node.js mit Express) mit eigenen Endpunkten. Ressourcenorientierte URLs, passende Methoden und **Statuscodes**, mindestens lesende und schreibende Endpunkte. Kurze API-Übersicht im README.
Schichten	Erkennbare Trennung von Routen, Logik und Datenzugriff.
Datenbank	Persistente Datenhaltung (z. B. SQLite oder PostgreSQL), Schema mit Migrationen oder ORM. Kein `data.json` als Fake-Datenbank.
Frontend-Anbindung	Fetch gegen das eigene Backend mit sichtbaren Lade- und Fehlerzuständen.
Authentifizierung	Registrierung und Login mit **Session oder JWT**, eine Variante konsequent. Passwörter gehasht gespeichert.
Autorisierung	Mindestens **zwei Rollen** mit unterschiedlichen Rechten. Die Prüfung passiert **im Backend**, nicht nur im Frontend.
Security-Check	Eingaben werden im Backend validiert. README-Abschnitt: Wie schützt eure App sich gegen XSS, Injection, CSRF bzw. CORS-Fehlkonfiguration? Wo liegen Secrets? Was ist bewusst noch offen?

Ergebnis: Eine Full-Stack-App mit eigener API, Datenbank, echten Accounts und Rollen und einer nachvollziehbaren Sicherheitsbetrachtung.

M4 · Abschluss

Abgabe: nach Vorlesungsende, Termin wird angekündigt · **VL-Bezug:** VL 13 bis 15

Kriterium	Beschreibung
Tests	Aussagekräftige Tests dort, wo das **Risiko** liegt (z. B. Autorisierung, Kernlogik).
Betrieb	Die App ist reproduzierbar startbar: Eine fremde Person bringt sie nach README in unter zehn Minuten zum Laufen.
Demo-Video	3 bis 5 Minuten Screencast: Registrierung, Login, Kernfunktionen, Rollen. Link im README.
Ausarbeitung	PDF, ca. 20 Seiten, siehe unten.

Ausarbeitung (mit M4)

Keine wissenschaftliche Arbeit, sondern eine nachvollziehbare Projektdokumentation:

Kapitel	Inhalt	Umfang
1. Einleitung	Projektidee, Zielgruppe, Rollen	ca. 1–2 Seiten
2. Technologie-Stack	Eingesetzte Technologien und warum	ca. 2 Seiten
3. Architektur	Komponenten, API-Design, Datenmodell, Diagramme erwünscht	ca. 3–4 Seiten
4. Umsetzung pro Meilenstein	Welche VL-Konzepte stecken wo im Code?	ca. 6–8 Seiten
5. Sicherheit	Authentifizierung, Autorisierung, Security-Check	ca. 2 Seiten
6. Tests und Betrieb	Teststrategie, wie die App gestartet wird	ca. 2 Seiten
7. Reflexion	Was lief gut, was würdet ihr anders machen?	ca. 2 Seiten
Anhang	Screenshots der wichtigsten Ansichten	

Schein und Note

Leistung	Grundlage	Ergebnis
Schein	alle vier **Labortestate** bestanden (bei mindestens 80 % Anwesenheit entfällt eines)	bestanden / nicht bestanden
Note	Semesterprojekt: alle vier Meilensteine abgegeben, bewertet werden Anwendung und Ausarbeitung über **6 Dimensionen**	benotet

Die Bewertungshandreichung mit den 6 Dimensionen findet ihr in Moodle unter „Allgemeines“.

Regeln

- **Einzelarbeit.** Austausch und gegenseitige Hilfe sind erwünscht, der Code eurer App ist aber eure eigene Arbeit.
- **KI-Werkzeuge sind erlaubt.** Ihr müsst jede Zeile eurer App erklären können.
- **Keine Secrets im Repository.** Keys und Passwörter gehören in `.env`, die nie committet wird. Testzugänge ins README oder in die Abgabenotiz.
- **Ein Repository** durchgängig über alle Meilensteine.